Лабораторная работа №9. Тестирование в CI/CD и анализ качества кода

1. Цель работы

Изучить принципы интеграции процесса тестирования в конвейер непрерывной интеграции и доставки (CI/CD), освоить инструменты статического и динамического анализа кода, а также научиться настраивать автоматическую генерацию отчетов о качестве и покрытии кода тестами.

2. Теоретическая часть

2.1. Непрерывная интеграция и непрерывная доставка (CI/CD)

CI/CD (Continuous Integration / Continuous Delivery) — это практика DevOps, которая автоматизирует этапы сборки, тестирования и развертывания приложений.

Основные концепции CI/CD:

- **Непрерывная интеграция (CI):** Разработчики регулярно (несколько раз в день) сливают свои изменения в центральный репозиторий, после чего автоматически запускается сборка и тестирование проекта. Это позволяет обнаруживать конфликты и ошибки на самых ранних этапах.
- **Непрерывная доставка (CD):** Расширение CI, при котором код после автоматического тестирования автоматически подготавливается к релизу и может быть развернут вручную в любой момент.
- **Непрерывное развертывание (CD):** Еще один шаг автоматизации, при котором каждое изменение, прошедшее все стадии тестирования, автоматически развертывается в production.

Ключевые компоненты CI/CD-пайплайна:

- **Pipeline (Пайплайн):** Набор автоматизированных задач (jobs), объединенных в стадии (stages), которые выполняются в определенном порядке (например, test → build → deploy).
- **Job (Задача):** Минимальная единица выполнения в пайплайне, содержащая скрипты для выполнения (например, запуск тестов, сборка проекта).
- **Stage (Стадия):** Группа задач, которые могут выполняться параллельно. Стадии выполняются последовательно.
- **Runner / Agent:** Агент, который выполняет задачи пайплайна (например, GitHub Actions runner, GitLab Runner) .
- **Artifacts (Артефакты):** Файлы, создаваемые в ходе выполнения задач (например, отчеты о тестировании, собранные DLL-файлы), которые сохраняются и могут быть переданы между стадиями или скачаны для анализа .
- **Cache (Кэш):** Механизм кэширования зависимостей (например, пакетов NuGet, node_modules) для ускорения последующих сборок .

Популярные CI/CD-платформы:

- **GitHub Actions:** Встроенная в GitHub система автоматизации. Пайплайны описываются в YAML-файлах в директории .github/workflows/ .
- **GitLab CI/CD:** Встроенная в GitLab система. Пайплайны описываются в файле .gitlab-ci.yml в корне репозитория .
- **Jenkins:** Самостоятельный сервер автоматизации с широкими возможностями настройки через плагины .

2.2. Роль тестирования в CI/CD пайплайне

В современной разработке тестирование является неотъемлемой частью CI/CD-конвейера. При каждом изменении кода автоматически запускаются различные виды тестов :

- **Модульные тесты (Unit Tests):** Запускаются первыми для быстрой проверки отдельных компонентов.
- **Интеграционные тесты (Integration Tests):** Проверяют взаимодействие модулей между собой и с внешними системами.
- **Линтинг и статический анализ (Linting & Static Analysis):** Проверяют соответствие кода стандартам качества и выявляют потенциальные ошибки без выполнения кода .

Интеграция проверок с pull request'ами: CI/CD позволяет настроить защиту веток (branch protection rules), при которой слияние pull request'a невозможно, пока все проверки качества (линтеры, тесты) не будут успешно пройдены . Это гарантирует, что в основную ветку попадает только качественный код.

2.3. Статический анализ кода

Статический анализ — это процесс оценки качества кода без его фактического выполнения. Инструменты статического анализа проверяют исходный код на наличие ошибок, нарушений стандартов кодирования, уязвимостей и "запахов кода" (code smells) .

Основные типы проблем, выявляемых статическим анализом :

- Нарушения стиля кодирования (отступы, именование переменных).
- Потенциальные логические ошибки (присваивание вместо сравнения).
- Неиспользуемые переменные и мертвый код.
- Дублирование кода.
- Сложные и трудно поддерживаемые конструкции (высокая цикломатическая сложность).
- Потенциальные уязвимости безопасности.

Популярные инструменты статического анализа:

- **SonarQube / SonarCloud:** Платформа для непрерывного анализа качества и безопасности кода. Поддерживает множество языков и предоставляет детальные отчеты с визуализацией метрик .
- **StyleCop / StyleCop.Analyzers:** Анализатор для C#, который проверяет соответствие кода правилам стиля и соглашениям о форматировании .
- **ReSharper:** Коммерческое расширение для Visual Studio с мощными возможностями анализа кода и рефакторинга .
- **SpotBugs / SpotLess:** Инструменты для статического анализа Java-кода .
- **CodeQL:** Анализатор безопасности от GitHub, встроенный в GitHub Actions .
- **PHPStan / Rector:** Инструменты для статического анализа PHP-кода .

Цикломатическая сложность (Cyclomatic Complexity): Это метрика, которая измеряет сложность программы на основе количества линейно независимых путей в коде. Высокая цикломатическая сложность указывает на то, что код трудно тестировать и поддерживать. Инструменты вроде **Lizard** или **xLizard** позволяют вычислять эту и другие метрики .

2.4. Анализ покрытия кода тестами

Покрытие кода (code coverage) — это метрика, показывающая, какая часть исходного кода выполняется при запуске тестов. Высокое покрытие не гарантирует отсутствие ошибок, но является хорошим индикатором того, что код протестирован .

Виды покрытия кода:

- Покрытие строк (Line Coverage): Сколько строк кода было выполнено.
- Покрытие ветвлений (Branch Coverage): Сколько веток условий (if-else) было выполнено.

Инструменты для измерения покрытия:

- **Coverlet:** Кроссплатформенная библиотека для .NET, которая собирает данные о покрытии кода .
- **JaCoCo (Java Code Coverage):** Популярный инструмент для измерения покрытия кода в Java-проектах .
- **Allure Reports:** Фреймворк для создания наглядных и интерактивных отчетов о выполнении тестов. Может агрегировать данные о покрытии и результаты тестирования в единый красивый отчет .

Материалы для самостоятельного изучения:

1. Непрерывная интеграция и непрерывная доставка (CI/CD): https://skillbox.ru/media/code/cicd-chto-eto-i-pochemu-bez-nee-ne-obhoditsya-razrabotka-po/ .

2. Настраиваем CI/CD с GitHub Actions и werf: инструкция для новичков: https://habr.com/ru/companies/flant/articles/803251/.

3. GitHub запустил статический анализ кода на уязвимости: https://habr.com/ru/companies/dcmiran/articles/521578/.

4. С Как настроить CI/CD с GitHub Actions: https://adminvps.ru/blog/kak-nastroit-ci-cd-s-github-actions/.

3. Задание на лабораторную работу

Работа выполняется на основе приложения, разработанного в предыдущих лабораторных работах. Вам предстоит настроить автоматизированный конвейер, который будет проверять качество кода и запускать тесты при каждом изменении в репозитории.

Задание 1. Статический анализ кода

1. **Выбор инструментов:** В зависимости от языка вашего приложения выберите инструменты для статического анализа. Рекомендуемый набор:
   - **Анализатор стиля:** StyleCop.Analyzers (C#), ESLint (JS/TS), PHP_CodeSniffer (PHP).
   - **Анализатор качества и безопасности:** SonarQube (локальный сервер) или SonarCloud (облачный).
   - **Анализатор сложности:** xLizard или Lizard для вычисления цикломатической сложности .
2. **Интеграция анализаторов в проект:**
   - Установите выбранные анализаторы как NuGet-пакеты или npm-пакеты в ваш проект.
   - Настройте правила анализа в соответствии с лучшими практиками (например, используйте стандартные наборы правил).
3. **Создание "неправильного" кода (опционально):** Если ваш код уже соответствует высоким стандартам качества, намеренно добавьте в него несколько типичных ошибок и нарушений стиля (как в примере ниже), чтобы проверить, смогут ли анализаторы их обнаружить.

```
// Пример кода с проблемами для C#
class Program
{
    static void Main(string[] args)
    {
        int n = 100;
        int i = 0;
        string s = "This is a string";
```

```
        // Деление на ноль (потенциальная ошибка)
        int result = n / i;

        // Присваивание вместо сравнения (логическая ошибка)
        if (n = 100)
        {
            Console.WriteLine("The value of n is 100.");
        }

        // Магическое число
        double area = 3.14159 * 7 * 7;
    }
}
```

Запуск анализа и исправление ошибок:

- Запустите локально все установленные анализаторы.
- Получите отчеты и проанализируйте найденные проблемы.
- Исправьте все критически важные замечания и "запахи кода". Добейтесь, чтобы код соответствовал стандартам качества.

Задание 2. Настройка CI/CD пайплайна

1. **Подготовка репозитория:**
   - Создайте репозиторий на GitHub (или GitLab) и загрузите в него свой проект, если вы этого еще не сделали.
2. **Создание CI/CD пайплайна:**
   - **Для GitHub Actions:** Создайте в корне репозитория директорию .github/workflows/ и добавьте в нее файл ci.yml .
   - **Для GitLab CI/CD:** Создайте в корне репозитория файл .gitlab-ci.yml .
3. **Конфигурация пайплайна:** Настройте пайплайн, который будет автоматически запускаться при push в любую ветку и при создании pull request. Пайплайн должен включать следующие стадии и задачи:

Стадия (Stage)	Задачи (Jobs)	Описание
test	lint, static-analysis, unit-tests	Проверка стиля, статический анализ, запуск модульных тестов (из ЛР №4).
integration	integration-tests	Запуск интеграционных тестов (из ЛР №5).
report	coverage, publish-reports	Генерация отчета о покрытии, публикация артефактов.

Пример фрагмента .gitlab-ci.yml **для стадии тестирования :**

```
stages:
```

```
  - test
  - integration
  - report

test-go:
  stage: test
  script:
    - go test ./... -v        # Запуск тестов
    - go vet ./...             # Статический анализ (аналог линтера)
  artifacts:
    paths:
      - coverage.out           # Сохранение отчета о покрытии
    reports:
      junit: report.xml         # Загрузка результатов тестов в формате JUnit
```

4. Проверка работы пайплайна: Сделайте коммит и пуш в репозиторий. Перейдите на вкладку "Actions" (GitHub) или "CI/CD → Pipelines" (GitLab) и убедитесь, что пайплайн запустился и все задачи выполнены успешно.

Задание 3. Анализ покрытия кода и генерация отчетов

1. Настройка сбора покрытия:

- Добавьте в проект инструмент для измерения покрытия: **Coverlet** для .NET , **JaCoCo** для Java или **pytest-cov** для Python.
- Настройте тесты так, чтобы при их выполнении генерировался отчет о покрытии.

2. Интеграция с CI/CD:

- Добавьте в пайплайн шаг для генерации отчета о покрытии.
- Настройте сохранение отчета как артефакта пайплайна .

3. Настройка порогов покрытия:

- Настройте задачу так, чтобы она завершалась ошибкой (fail), если покрытие кода падает ниже установленного порога (например, менее 70% для ключевых модулей User и AuthenticationService).

4. **Генерация Allure-отчетов (опционально, но рекомендуется) :**

- Добавьте в пайплайн шаги для генерации Allure-отчета на основе результатов тестов.
- Настройте публикацию Allure-отчета на GitHub Pages (или GitLab Pages), чтобы отчет был доступен по постоянной ссылке после каждого запуска пайплайна.

4. Содержание отчёта

Отчёт должен содержать:

1. **Цель работы.**
2. **Описание статического анализа (Задание 1):**
   - Перечень использованных инструментов статического анализа с обоснованием выбора.
   - Скриншоты отчётов анализаторов **до** исправления ошибок (с указанием найденных проблем).
   - Листинг исправленного кода с комментариями о том, какие проблемы были устранены.
   - Скриншоты отчётов **после** исправления (чистый результат).
3. **Описание CI/CD пайплайна (Задание 2):**
   - Полный листинг файла конфигурации CI/CD (.github/workflows/ci.yml или .gitlab-ci.yml) с подробными комментариями.
   - Скриншоты успешного выполнения пайплайна в репозитории (общий вид пайплайна и результаты каждой задачи).
   - Скриншот настроек защиты ветки (branch protection), где проверки CI/CD отмечены как обязательные для слияния .
4. **Описание анализа покрытия (Задание 3):**
   - Скриншот отчёта о покрытии кода тестами.
   - Анализ "белых пятен" (какие строки/методы остались непокрытыми и почему).
   - (Опционально) Ссылка на опубликованный Allure-отчет и его скриншот.
5. **Выводы:** Опишите, как внедрение CI/CD и автоматического анализа качества влияет на процесс разработки, скорость обнаружения дефектов и общую надежность программного продукта.

5. Контрольные вопросы

Основы CI/CD

1. Что такое CI/CD и каковы его основные компоненты?
2. Чем отличаются непрерывная доставка (Continuous Delivery) от непрерывного развертывания (Continuous Deployment)?
3. Что такое пайплайн (pipeline), стадия (stage) и задача (job)?
4. Какие преимущества дает интеграция автоматического тестирования в CI/CD?
5. Чем артефакты (artifacts) отличаются от кэша (cache) в CI/CD?

Статический анализ кода

6. Что такое статический анализ кода и какие проблемы он помогает выявить?

7. Чем статический анализ отличается от динамического (модульного тестирования)?

8. Что такое "запах кода" (code smell)? Приведите примеры.

9. Что такое цикломатическая сложность и почему важно ее контролировать?

10. Какие инструменты статического анализа для вашего языка программирования вы знаете?

Покрытие кода и отчетность

11. Что такое покрытие кода (code coverage) и какие его виды существуют?

12. Какие инструменты для измерения покрытия кода вы знаете?

13. Является ли 100% покрытие кода гарантией отсутствия ошибок? Почему?

14. Для чего используются Allure Reports?

15. Как настроить публикацию отчетов в GitHub Pages или GitLab Pages?

Интеграция и безопасность

16. Как настроить защиту ветки (branch protection), чтобы требовать успешного прохождения CI-проверок перед слиянием?

17. Что такое CodeQL и для чего он используется?

18. Как безопасно хранить чувствительные данные (токены, пароли) в CI/CD пайплайнах?

19. Что такое allow_failure в GitLab CI/CD и для чего он используется?

20. Как обеспечить быстродействие CI/CD пайплайнов (ускорение сборки и тестирования)?

6. Рекомендуемая литература и источники для самостоятельного изучения

6.1. Официальная документация и руководства

- **GitHub. "GitHub Actions Documentation" [Электронный ресурс] / GitHub Docs.**
  - **Доступ:** https://docs.github.com/en/actions.
  - **Аннотация:** Официальная документация по GitHub Actions. Содержит примеры создания пайплайнов, описания всех ключевых слов и событий, а также руководства по настройке тестирования для различных языков программирования.

- **GitLab. "GitLab CI/CD Documentation" [Электронный ресурс] / GitLab Docs.**
  - **Доступ:** https://docs.gitlab.com/ee/ci/
  - **Аннотация:** Полное руководство по GitLab CI/CD. Включает описание структуры .gitlab-ci.yml, настройку раннеров, работу с артефактами, окружениями и защитой веток .
- **Microsoft Learn. "Run linting and unit testing" [Электронный ресурс] / Microsoft Training.**
  - **Доступ:** https://learn.microsoft.com/en-ca/training/modules/work-linting-unit-test-github-actions/
  - **Аннотация:** Модуль обучения от Microsoft, который подробно объясняет, как настроить линтинг и модульное тестирование в GitHub Actions. Рассматриваются вопросы интеграции проверок с pull request'ами и настройки защиты веток .
- **SonarSource. "SonarQube Documentation" [Электронный ресурс] / SonarSource Docs.**
  - **Доступ:** https://docs.sonarqube.org/latest/
  - **Аннотация:** Официальная документация по установке и настройке SonarQube. Содержит руководства по интеграции с различными CI/CD-системами и языками программирования.

6.2. Учебные пособия и лабораторные работы

- **Toschyk. "CodeQualityLab" [Электронный ресурс] / GitHub.**
  - **Доступ:** https://github.com/Toschyk/CodeQualityLab
  - **Аннотация:** Пример лабораторной работы на русском языке, посвященной анализу качества кода C#. Содержит пример "неправильного" кода с типичными ошибками и подробные инструкции по работе с SonarQube, ReSharper, StyleCop и Coverlet .
- **mediana105. "software-testing-lab-3" [Электронный ресурс] / GitHub.**
  - **Доступ:** https://github.com/mediana105/software-testing-lab-3
  - **Аннотация:** Пример лабораторной работы по настройке CI/CD с GitHub Actions, генерации отчетов JaCoCo и Allure, а также подключению статического анализатора кода. Отличный практический пример для студентов .
- **ВГТУ. "Лаб. работа 'CI/CD с GitLab'" [Электронный ресурс] / it.vstu.by.**
  - **Доступ:** https://it.vstu.by/courses/information_systems/industrial_software_development_tools/practice/cicd/

- **Аннотация:** Подробное учебное пособие на русском языке по настройке CI/CD в GitLab. Содержит теоретическую часть с объяснением ключевых концепций и практическую часть с пошаговыми инструкциями по созданию пайплайна, работе с артефактами и настройке окружений .

6.3. Книги и курсы

- **Willett, J. "CI/CD for test automation: Jenkins & GitHub actions" [Электронный ресурс] / Packt Publishing, 2025.**
  - **Доступ:** https://www.oreilly.com/library/view/cicd-for-test/9781806385614/
  - **Аннотация:** Видеокурс, охватывающий настройку Jenkins и GitHub Actions для автоматизации тестирования. Рассматриваются стратегии запуска unit, API, E2E и performance-тестов, а также публикация результатов тестирования .
- **xLizard. "Extended Cyclomatic Complexity Analyzer" [Электронный ресурс] / PyPI.**
  - **Доступ:** https://pypi.org/project/xlizard/
  - **Аннотация:** Документация к инструменту xLizard для анализа цикломатической сложности кода. Полезно для понимания метрик сложности и их визуализации в HTML-отчетах .